

the maximum accuracy on the validation set. We can easily and concisely express this problem in DeepStochLog as follows:

```

1 class(Y) :- member(Y, [class0, class1, ..., class6]).
2 nn(classifier, [X], [Y], [class]):: doc_neural(X,Y) --> [].
3
4 1/Na :: cite(a,b) --> [].
5 1/Na :: cite(a,c) --> [].
6 1/Nb :: cite(b,d) --> [].
7 ...
8
9 0.5:: doc(X,Y) --> doc_neural(X,Y).
10 0.5:: doc(X,Y) --> cite(X, X1), doc(X1,Y).
11
12 s(X) --> doc(X,Y), [Y].

```

The program simply states that a document X is of class Y (and produce the terminal Y) either if a neural network classifies it so or if it is cited by one or more documents of the same class. Even if strange, many of these citation networks are actually cyclic. Therefore, we limited the depth of the derivations to a maximum value to avoid having a cyclical program. We also experimented with a variant of the program in which the probabilities of the rules expanding the `doc` predicate are trained separately for each class Y .

B.6 Word Algebra Problem

We trained the model on the Word Algebra Problem for 40 epochs using the Adam optimizer with a learning rate of 0.001 and a batch size of 32. The DeepStochLog program is then modelled as follows:

```

1 permute_d(Y) :- member(Y, [0,1,2,3,4,5]).
2 op(Y) :- member(Y, [plus,minus,times,div]).
3 swap_d(Y) :- member(Y, [no_swap, swap]).
4
5 nn(nn_permute, [X], [Y], [permute_d]):: nn_permute(X,Y) --> [].
6 nn(nn_op1, [X], [0], [op]):: nn_op1(X, 0) --> [].
7 nn(nn_swap, [X], [S], [swap_d]):: nn_swap(X, S) --> [].
8 nn(nn_op2, [X], [0], [op]):: nn_op2(X, 0) --> [].
9
10
11 0.1666::permute(0,A,B,C,A,B,C) --> [].
12 0.1666::permute(1,A,B,C,A,C,B) --> [].
13 0.1666::permute(2,A,B,C,B,A,C) --> [].
14 0.1666::permute(3,A,B,C,B,C,A) --> [].
15 0.1666::permute(4,A,B,C,C,A,B) --> [].
16 0.1666::permute(5,A,B,C,C,B,A) --> [].
17
18 0.5:: swap(no_swap,X,Y,X,Y) --> [].
19 0.5:: swap(swap,X,Y,Y,X) --> [].
20
21 0.25:: operator(plus,X,Y,Z) --> [], {Z is X+Y}.
22 0.25:: operator(minus,X,Y,Z) --> [], {Z is X-Y}.
23 0.25:: operator(times,X,Y,Z) --> [], {Z is X*Y}.
24 0.25:: operator(div,X,Y,Z) --> [], {Y > 0, 0 := X mod Y, Z is X//Y}.
25
26 s(Out,X1,X2,X3) --> [String],
27                     nn_permute(String, Perm),
28                     nn_op1(String, Op1),

```

```

29         nn_swap(String, Swap),
30         nn_op2(String, Op2),
31         permute(Perm,X1,X2,X3,N1,N2,N3),
32         operator(Op1,N1,N2,Res1),
33         swap(Swap,Res1,N3,X,Y),
34         operator(Op2,X,Y,Out).

```

B.7 Neural network architectures

Table 8 summarizes the neural network architectures used in the experiment. *Conv(o,k)* is a convolutional layer with o output channels and kernel size k . *Lin(n)* is a fully-connected layer of size n . *(Bi)GRU(h)* is a single-layer (bi-directional) GRU with a hidden size h . *MaxPool(k)* is a max-pooling layer with kernel size k , and *Dropout(p)* a dropout layer with probability p . A layer in bold means it is followed by a ReLU activation function. All neural networks end with a Softmax layer, unless otherwise specified.

Table 8: Overview of the neural network architectures used in the experiments.

Task	Network	Architecture
T1	number	MNISTConv, Lin(120) , Lin(84) , Lin(10)
T2	number	Conv(6, 3) , MaxPool(2), Conv(16,3) , MaxPool(2), Dropout(0.4), Lin(128) , Lin(10)
	operator	Conv(6, 3) , MaxPool(2), Conv(16,3) , MaxPool(2), Dropout(0.4), Lin(128) , Lin(4)
T3	bracket_nn	MNISTConv, Lin(120) , Lin(84) , Lin(2)
T4	mnist	MNISTConv, Lin(120) , Lin(84) , Lin(3)
T5	classifier-Citeseer	Lin(50) , Lin(6)
	classifier-Cora	Lin(50) , Lin(7)
T6	RNN	Embedding(256), BiGRU(512), Dropout(0.5)*
	nn_permute	Lin(6)
	nn_op1	Lin(4)
	nn_swap	Lin(2)
	nn_op2	Lin(4)

MNISTConv: Conv(6,5), **MP(2,2)**, Conv(16,5), **MP(2,2)***
AlexNetConv: **Conv(64, 11, 2,2)**, MP(3,2), **Conv(192, 5, 2)**, MP(3,2),
Conv(384, 3, 1), **Conv(256, 3, 1)**, **Conv(256, 3, 1)**, MP(3,2)*

* Does not end with a Softmax layer.

C Translation Example of Neural Definite Clause Grammar to Prolog

Consider the following DeepStochLog program.

```

1 digit(Y) :- member(Y, [0,1,2,3,4,5,6,7,8,9]).
2 op(Y) :- member(Y, [+,-]).
3
4 nn(mnist,[I],[N],[digit]) :: n(N) --> [I].
5 nn(operator,[I],[N],[op]) :: o(N) --> [I].
6
7 0.33::e(N) --> n(N).
8 0.33::e(S) --> e(E1), o(+), n(E2), {S is E1 + E2}.
9 0.33::e(S) --> e(E1), o(-), n(E2), {S is E1 - E2}.

```

It is translated into:

```

1 digit(Y) :- member(Y, [0,1,2,3,4,5,6,7,8,9]).
2 op(Y) :- member(Y, [+,-]).
3
4 n(N, [I | X], [X]) :- nn(mnist,[I],[N]), digit(X).
5 n(N, [I | X], [X]) :- nn(mnist,[I],[0]), op(0).
6
7 e(N, A, B) :- n(N, A, B), p(0.33).
8 e(S, A,D) :- e(E1, A, B), o(+, B, C), n(E2, C, D), S is E1 + E2, p(0.33).
9 e(S, A,D) :- e(E1, A, B), o(-, B, C), n(E2, C, D), S is E1 - E2, p(0.33).

```

Both the calls to `nn` and `p` are considered always `true` during logical inference. During evaluation of the arithmetic circuit, `nn(mnist,[I],[0])` returns the probability of the output `0` when provided with the image `I` as input, while `p(x)` constantly returns the probability `x`. It is easy to see that any derivation of the Prolog program always end with either a `nn` or a `p` call, which constitute the leaves of the correspondent arithmetic circuit.